



Data Pipelines | HW 04

Выполнил: Васюков Александр, avvasiukov@edu.hse.ru

Github: <https://github.com/vasyukov1/hse-data-pipelines/tree/main/hw04>

Задание

ФОРМАТ

Вы — архитектор данных. К вам приходит джуниор: “Я буду все писать на DataFrame, это современно”.

ЗАДАЧА

Придумать задачу, на которой докажете, что RDD:

- Компактнее — меньше кода
- Быстрее — ручные оптимизации добавляют скорость
- Надежнее — возможность управлять памятью
- Гибче — позволяют сделать то, для чего DataFrame не очень подходит

УСЛОВИЕ

Задача должна быть реалистичной.

Решение

Описание задачи

Нужно обнаруживать “опасные атаки” в трекингowych данных футбольных матчей — короткие, быстрые эпизоды продвижения мяча сразу после перехвата. Это не простая агрегация: требуется анализ последовательности кадров, геометрии позиций и состояния игроков в виде конечного автомата. Задача ориентирована на алгоритмическую обработку потоков, где важен контроль порядка и состояния внутри матча.

Входные данные

Каждые 40 ms:

- координаты 22 игроков
- координаты мяча
- скорости
- владение
- timestamp

Итого, один матч = десятки миллионов кадров.

Формат — бинарные логи (protobuf), данные распределены по 200+ матчам и не отсортированы глобально.

Эти кадры нужны для восстановления траекторий, вычисления пройденного пути и переходов состояний (перехват → контратака → завершение).

Что хотим получить

На выходе — список эпизодов с метриками:

- start/end time
- пройденные метры
- число передач
- средняя скорость
- состав игроков

Формат — компактная таблица эпизодов (Parquet) и сериализованные фичи для обучения ML-моделей (xG) и для аналитики.

Потребители: аналитики клубов, тренеры и модели машинного обучения — то есть результат должен быть точным, детерминированным и быстро доступным.

Объём данных и хранение

Сырые логи — ТБ в HDFS/S3; агрегаты — десятки ГБ. При таком объёме критичны локальность вычислений и минимизация shuffle.

Нужно хранить по match_id/дате, уметь читать только нужные файлы и избегать лишних промежуточных форматов.

Если гонять ТБ по сети ради одной функции — производительность и бюджет пострадают одновременно.

Почему DataFrame неудобен

Stateful-логика:

Сложную логику со сменой состояний и проверками на каждом шаге проще и понятнее реализовать обычным алгоритмом в коде. В DataFrame такую логику приходится собирать из множества оконных функций и пользовательских функций, из-за чего появляется много промежуточных колонок, shuffle-операций и трудно читаемый пайплайн, который сложнее отлаживать и контролировать по производительности.

Контроль партиций:

Нужна гарантия “весь матч в одной партиции” для корректного анализа; DataFrame не даёт такой тонкой ручки управления.

Кастомные структуры:

Кадр — вложенный объект с геометрией; расплющивание в колонки усложнит код и увеличит сериализацию.

Производительность:

Window + shuffle на ТБ данных — дорого. DataFrame красиво выражает запрос, но часто скрывает высокую стоимость shuffle.

DataFrame ориентирован на табличные операции и оптимизации Catalyst; он хорош для SQL-агрегаций, но не даёт тонкого контроля над

последовательностями событий и партиционированием по матчу. Реализация `stateful sliding-window` с кастомной логикой (конечный автомат, проверки геометрии на каждом кадре) превращается в громоздкие пользовательские функции/сериализации или в частые `shuffle`-операции. Кроме того, схемы колонок не подходят для хранения и быстрого обхода сложных вложенных объектов кадра без дополнительных преобразований.

Почему выгодно использовать RDD

Компактнее — меньше кода

Парсим бинарник в объекты `Frame` и пишем алгоритм автомата в `mapPartitions`.

Одна функция — одна задача: немного строк читабельного кода вместо десятков UDF и сложных SQL-окон.

Код близок к алгоритму — легче тестировать и деплоить.

Быстрее — ручные оптимизации дают эффект

`partitionBy(match_id)` + локальная сортировка = матч обрабатывается в памяти одной партии, почти без `shuffle`.

`mapPartitions` читает и обрабатывает кадры линейно, минимизируя сетевой трафик и сериализацию.

На практике это даёт заметный выигрыш на терабайтах: меньше GC, меньше I/O, меньше ожиданий.

Надёжнее — контроль памяти и `fault-tolerance`

RDD хранит `lineage`: при падении пересчитывается только затронутая партия (например, один матч), а не весь датафрейм.

Мы явно решаем, что кешировать и где делать `checkpoint` — это уменьшает вероятность неожиданного `Out of Memory`.

Такая управляемая надёжность важно для `production`-пайплайнов.

Гибче — сложная логика “из коробки”

В `mapPartitions` держим состояние автомата, работаем с геометрией и пишем любые проверки на лету.

Это позволяет реализовать нестандартные правила (например, учитывать направление векторов, плотность обороны).

Для задач, где “как” важнее чем “что”, RDD — естественный инструмент.

Итоговая архитектура пайплайна

1. `binaryFile()` → RDD[bytes] — читаем как поток байт, без лишних преобразований.
2. `parse` → RDD[Frame] — получаем нативные объекты кадра с геометрией.
3. `partitionBy(match_id)` + локальная сортировка — гарантируем локальность и порядок.
4. `mapPartitions(detect_attacks)` — конечный автомат выделяет эпизоды на лету.
5. `reduceByKey` / `aggregate` — собираем командные метрики.
6. `saveAsParquet` — итог в удобном формате для BI/ML.

Каждый шаг минимизирует shuffle и делает пайплайн детерминированным и отлаживаемым.

Вывод

DataFrame — отличный инструмент для декларативной аналитики и отчётов. Но когда задача — stateful, последовательная, геометрическая и алгоритмическая, RDD даёт явный контроль: код короче, выполнение быстрее, поведение предсказуемое и гибче для нестандартной логики.